

# Ex03

## Answers & Manual

Paul Tröger, Fynn Gohlke, Tom Rodewald, Jan Büchele

April 28, 2026

## 1 Answers

### 1.1 Shamir secret sharing

We assume that all parties share the same known  $x$  evaluation points for shamir secret sharing. In our code we draw these from a fixed seed with `np.random.choice`. This is necessary for adding shares together.

### 1.2 Ben-Or, Goldwasser, and Wigderson (BGW) Protocol

#### Assumption:

We assume that all parties share the same known  $x$  evaluation points for shamir secret sharing. In our code we draw these from a fixed seed with `np.random.choice`. Each party is responsible for evaluation point  $x_p$  where  $p$  is the party id.

#### Protocol:

1. We generate shamir secrets for  $a, b$  with  $t, n$  getting  $n$  shares per secret.
2. We initialize  $n$  parties so that each party gets **one share per secret**.
3. We start round 1 by handing out shares:
  - (a) Each party multiplies it's secret shares  $y$  values, increasing the polynomials degree to  $2t$ .
  - (b) Each party generates a new shamir secret where  $q(x_p) = p_\alpha(x_p) * p_\beta(x_p)$ .
  - (c) Each party sends out  $n$  shares for  $q(x_p)$  to the  $n$  parties.
4. Each party now holds  $n$  shares for their assigned evaluation point  $x_p$ .
5. We start round 2:
  - (a) Each party calculates Lagrange coefficients as described in 1.2.1.
  - (b) Each party calculates  $q(0) = \sum_{i=1}^{2t+1} \lambda_i q(i)$  as described in "A Pragmatic Introduction to Secure Multi-Party Computation" for each party and store it in the output variable.
6. We retrieve the output from all parties with `get_view()` and reconstruct the value.

#### 1.2.1 Lagrange Coefficients

The Lagrange Coefficients are defined as

$$L_k(x) = \prod_{i=0, i \neq k}^n \frac{(x - x_i)}{(x_k - x_i)}$$

with properties  $L_k(x_k) = 1$  and  $L_k(x_j) = 0$  where  $x_j \neq x_k$ .

## 2 Manual

### 2.1 Running the code

All source code is self contained within a singular file called "e3.py".

Running the application can be achieved using *python3 e3.py*.

This runs all the tasks and their respective outputs, Listing 3.1 shows the output of running the application normally.

There are also python tests defined within the application. These can be run using *pytest -v e3.py*.

The flag "-v" is not mandatory, but improves clarity on what actually happened.

Running the tests with the aforementioned command results in an output similar to Listing 3.2.

### 2.2 Test Cases

#### 2.2.1 Task 1 Tests

For the `shamir_share(x, t, n)` function we test if the reconstruction is possible in for  $t + 1$  shares. We test if reconstruction is not possible if the shares are insufficient. We also test that all share points are unique. For `add_const` and `mult_const` we test a simple case and a wrap around in the field space.

#### 2.2.2 Task 2 Tests

We wrote two tests for `run_bgw(t, n, a, b)` which also tests BGW class.

In test one `test_run_bgw_large_t()` we check that the assert inside `round1()` throws an exception if  $2t+1 \leq n$ .

In test two `test_run_bgw_correctness()` we test different configurations and their output.

We test `get_lagrange_coeffs(x_values)` with a two point and three point example. We can calculate the result per hand and verify if our function outputs correct Lagrange coefficients. See 1.2.1 for Lagrange equation.

## 3 Appendix

### 3.1 Python-output

```
————— Task 1 —————
t:1, n:4, a:12, b:5
shares_a
[[1557 6833]
 [6987 5401]
 [6795  709]
 [6474 5663]]
shares_b
[[1557 6833]
 [6987 5401]
 [6795  709]
 [6474 5663]]
Adding constant 2 to shares_a
re: 14
Adding shares together..
re: 19
```

```
————— Task 2 —————
t:2, n:5, a:5, b:5
re: 25
```

### 3.2 Pytest-output

```
===== test session starts =====
platform linux — Python 3.14.3, pytest-9.0.2, pluggy-1.5.0
cachedir: .pytest_cache
collecting ... collected 11 items

e3.py::test_shamir_share_reconstruction PASSED [ 9%]
e3.py::test_shamir_share_insufficient_shares PASSED [ 18%]
e3.py::test_shamir_share_uniqueness PASSED [ 27%]
e3.py::test_add_const PASSED [ 36%]
e3.py::test_add_const_field_wraparound PASSED [ 45%]
e3.py::test_mult_const PASSED [ 54%]
e3.py::test_mult_const_field_wraparound PASSED [ 63%]
e3.py::test_run_bgw_large_t PASSED [ 72%]
e3.py::test_run_bgw_correctness PASSED [ 81%]
e3.py::test_get_lagrange_coeffs_two_points PASSED [ 90%]
e3.py::test_get_lagrange_coeffs_three_points PASSED [100%]

===== 11 passed in 4.24s =====
```